

3 • Universal Differential Equations (UDE)

A **Universal Differential Equation** mixes *known* mechanistic terms with an *unknown* term represented by a neural network, and learns the unknown part from data. This is the key idea behind Rackauckas et al. (2020), *Universal Differential Equations for Scientific Machine Learning*.

We use the Lotka–Volterra predator–prey model:

$$\dot{x} = \alpha x - \beta xy, \quad \dot{y} = -\gamma y + \delta xy$$

We pretend we only know the growth term αx and decay term $-\gamma y$, and let a neural network $U_\theta(x, y)$ learn the *interaction* terms $-\beta xy$ and δxy directly from trajectory data.

```
1 using Lux, OrdinaryDiffEq, SciMLSensitivity, Optimization, OptimizationOptimisers,  
   ComponentArrays, Random, Plots
```

1. Ground-truth data from the full Lotka–Volterra model

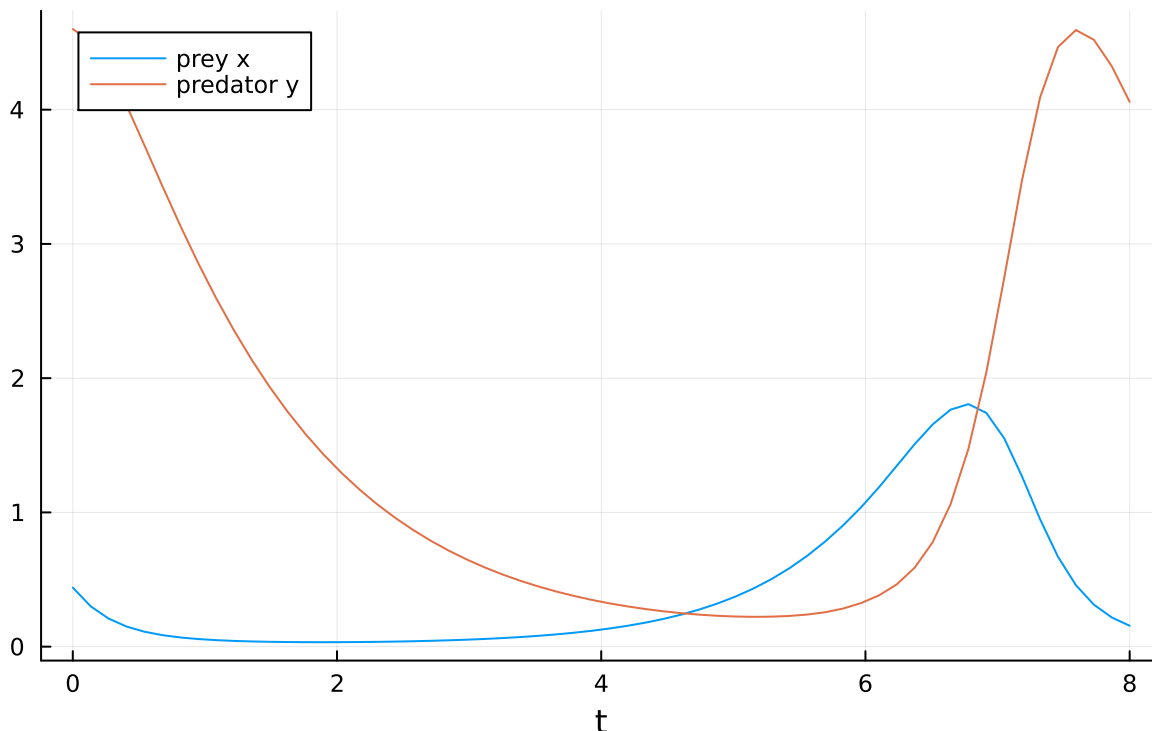
lotka_volterra! (generic function with 1 method)

```
1 function lotka_volterra!(du, u, p, t)  
2     x, y = u  
3     α, β, γ, δ = p  
4     du[1] = α * x - β * x * y  
5     du[2] = -γ * y + δ * x * y  
6     return nothing  
7 end
```

```
2×60 Matrix{Float64}:
0.44  0.300511  0.209113  0.1499  ...  0.457926  0.313321  0.217599  0.155526
4.6   4.51223   4.30469   4.03305  ...  4.59314   4.52083   4.32416   4.05814
```

```
1 begin
2   rng3 = Random.default_rng()
3   Random.seed!(rng3, 42)
4
5   true_p = (1.3, 0.9, 0.8, 1.8) # α, β, γ, δ
6   u0_lv = [0.44, 4.6]
7   tspan_lv = (0.0, 8.0)
8   tsteps_lv = range(tspan_lv[1], tspan_lv[2]; length=60)
9
10  lv_prob = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, true_p)
11  lv_sol = solve(lv_prob, Tsit5(); saveat=tsteps_lv)
12  lv_data = Array(lv_sol)
13 end
```

Lotka-Volterra ground truth



```
1 plot(tsteps_lv, lv_data'; label=["prey x" "predator y"], xlabel="t", title="Lotka-
Volterra ground truth")
```

2. The hybrid (universal) model ⇔

We keep the known linear growth/decay terms αx and $-\gamma y$ exactly, and replace the *unknown* interaction terms with a small neural network $U_\theta(x, y) \in \mathbb{R}^2$:

$$\dot{x} = \alpha x + U_\theta(x, y)_1, \quad \dot{y} = -\gamma y + U_\theta(x, y)_2$$

```
ComponentVector{Float32}(layer_1 = (weight = Float32[0.025153046 0.4572587; -1.7960389 -0.80;
```

```
1 begin
2     interaction_nn = Chain(Dense(2, 8, tanh), Dense(8, 8, tanh), Dense(8, 2))
3     nn_ps_init, nn_st = Lux.setup(rng3, interaction_nn)
4     nn_ps_init = ComponentArray(nn_ps_init)
5 end
```

0.8

```
1 begin
2     # only the growth/decay rates are assumed known
3      $\alpha_{\text{known}} = 1.3$ 
4      $y_{\text{known}} = 0.8$ 
5 end
```

ude_dynamics! (generic function with 1 method)

```
1 function ude_dynamics!(du, u, p, t)
2     x, y = u
3      $\hat{u}, \_ = \text{interaction\_nn}(u, p, \text{nn\_st})$ 
4      $\text{du}[1] = \alpha_{\text{known}} * x + \hat{u}[1]$ 
5      $\text{du}[2] = -y_{\text{known}} * y + \hat{u}[2]$ 
6     return nothing
7 end
```

```
ude_prob = ODEProblem with uType Vector{Float64} and tType Float64. In-place: true
Non-trivial mass matrix: false
timespan: (0.0, 8.0)
u0: 2-element Vector{Float64}:
 0.44
 4.6
```

```
1 ude_prob = ODEProblem(ude_dynamics!, u0_lv, tspan_lv)
```

3. Train the neural interaction term against the data ⇔

predict_ude (generic function with 1 method)

```
1 function predict_ude(p)
2     solve(ude_prob, Tsit5(); p=p, saveat=tsteps_lv,
3         sensealg=InterpolatingAdjoint(; autojacvec=ZygoteVJP())) |> Array
4 end
```

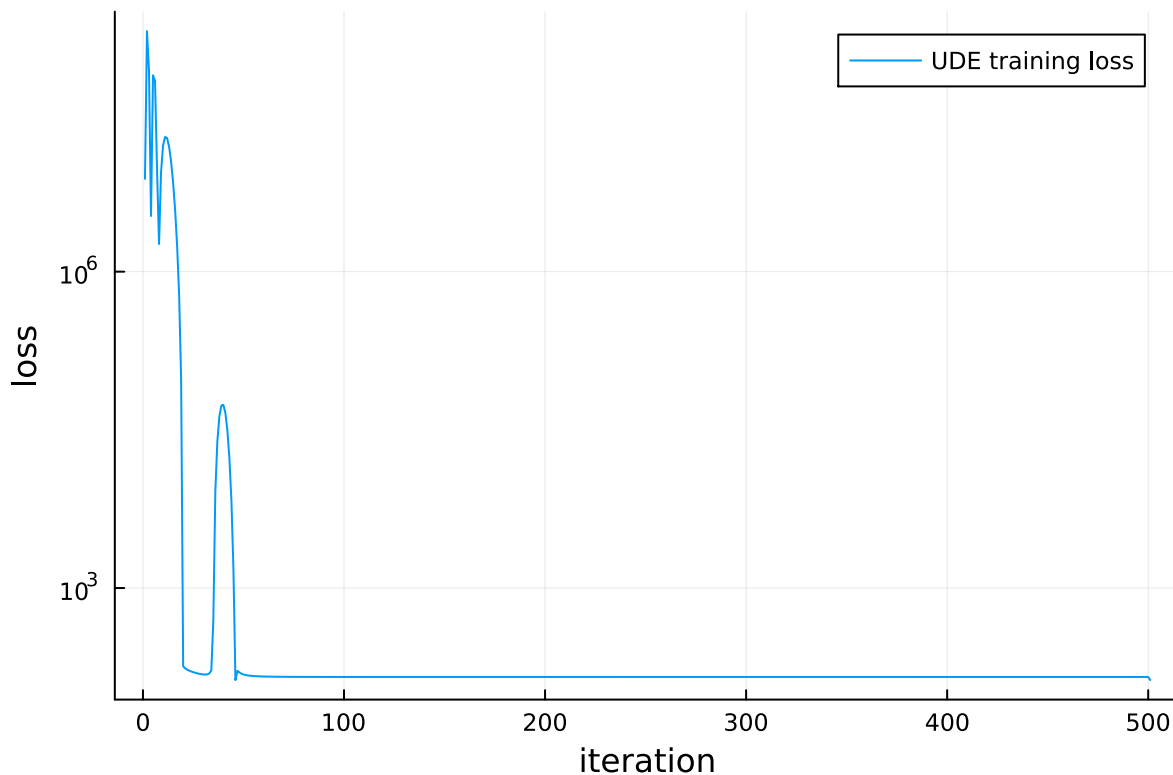
loss_ude (generic function with 1 method)

```
1 function loss_ude(p, _)
2     pred = predict_ude(p)
3     size(pred) == size(lv_data) || return Inf
4     return sum(abs2, lv_data .- pred)
5 end
```

► SciMLBase.OptimizationSolution{Float32, 1, ComponentArrays.ComponentVector{Float32, Vector{Float32}}

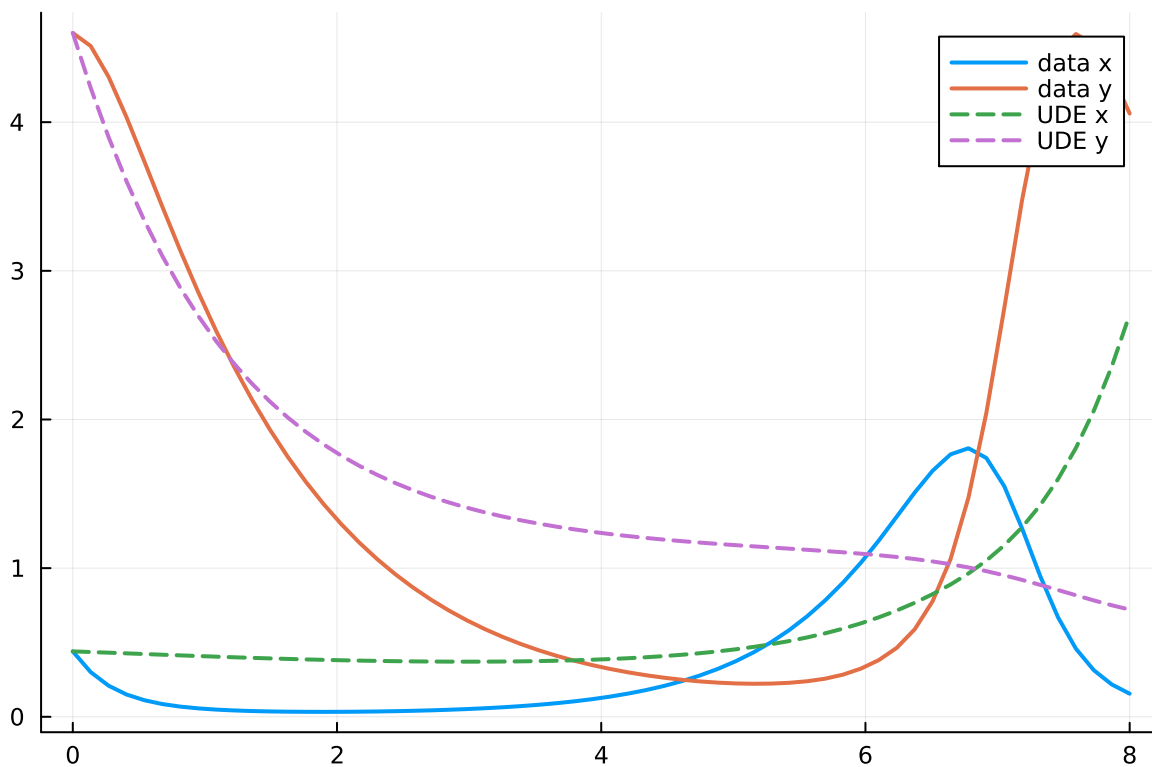
```
1 begin
2     adtype3 = Optimization.AutoZygote()
3     optf3 = OptimizationFunction(loss_ude, adtype3)
4     optprob3 = OptimizationProblem(optf3, nn_ps_init)
5
6     ude_losses = Float64[]
7     ude_callback = function (state, loss)
8         push!(ude_losses, loss)
9         return false
10    end
11
12    res_ude = Optimization.solve(optprob3, OptimizationOptimisers.Adam(0.02);
13    callback=ude_callback, maxiters=500)
14 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.



```
1 plot(ude_losses; xlabel="iteration", ylabel="loss", yscale=:log10, label="UDE training loss")
```

4. Does the learned model reproduce the trajectories? ⇌



```
1 begin
2     ude_fit = predict_ude(res_ude.u)
3     plot(tsteps_lv, lv_data'; label=["data x" "data y"], lw=2)
4     plot!(tsteps_lv, ude_fit'; label=["UDE x" "UDE y"], lw=2, ls=:dash)
5 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.

5. Inspect the learned interaction term ⇌

The true missing interaction terms are $-\beta xy$ and δxy . We can compare them against what the trained network U_θ actually learned, on a grid of (x, y) values.

```
► (learned = [-0.701883, 0.426179], true_term = [-1.35, 2.7])
```

```
1 begin
2   test_x, test_y = 1.5, 1.0
3   learned_interaction, _ = interaction_nn([test_x, test_y], res_ude.u, nn_st)
4   true_interaction = [-true_p[2] * test_x * test_y, true_p[4] * test_x * test_y]
5   (learned=learned_interaction, true_term=true_interaction)
6 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.

Takeaways ↪

- Only the physics you *don't* trust needs to be replaced by a neural network — everything you already know (conservation laws, known rates) stays exactly as an equation.
- Because the unknown term is low-dimensional and structured, the UDE typically needs far less data than a fully black-box neural ODE.
- Once U_θ is trained, its input/output pairs can be fed to a **sparse regression** method (SINDy, see [06_sindy_model_discovery.jl](#)) to recover a symbolic formula such as $-\beta xy$ — turning the learned network back into an interpretable equation.

